

Name: Mark Vincent Anas

Date: 05-04-26

Instructor: Jeremiah Dela Cruz

Individual Activity4: “Design Your Own System Using UML”

Part 1: UML Recall and System Idea

1. At least 4 UML Diagrams with brief descriptions:

- **Use Case Diagram:** This behavioral diagram illustrates the functional requirements of a system by showing interactions between external actors (users) and the various use cases (functions) the system provides. It helps identify who uses the system and what they can do.
- **Class Diagram:** A structural diagram that shows the static structure of the system, including classes, their attributes, methods, and relationships (inheritance, association, etc.). It serves as the blueprint for object-oriented implementation.
- **Activity Diagram:** This behavioral diagram represents the workflow or step-by-step activities and decisions within a process. It is similar to a flowchart and is useful for modeling business processes or complex operations.
- **Sequence Diagram:** A behavioral (interaction) diagram that shows how objects interact in a specific sequence over time, emphasizing the order of messages exchanged between actors and system components.

System Title: Inventory Management System

Short Description: The Inventory Management System is a software application designed to help businesses efficiently track, manage, and control their stock levels in real-time. It allows users to monitor incoming and outgoing items, reduce stockouts or overstock situations, and generate reports for better decision-making. The system supports multiple users such as warehouse staff and managers, ensuring accurate inventory records and streamlined operations for small to medium businesses.

3–5 Main Features:

1. Add/Update/Delete inventory items (with details like product ID, name, quantity, price, supplier).
2. Real-time stock monitoring and low-stock alerts.
3. Record stock inflows (purchases/receipts) and outflows (sales/issues).
4. Generate inventory reports (stock summary, transaction history, value reports).
5. User authentication and role-based access (Admin, Manager, Staff).

Part 2: UML Diagrams

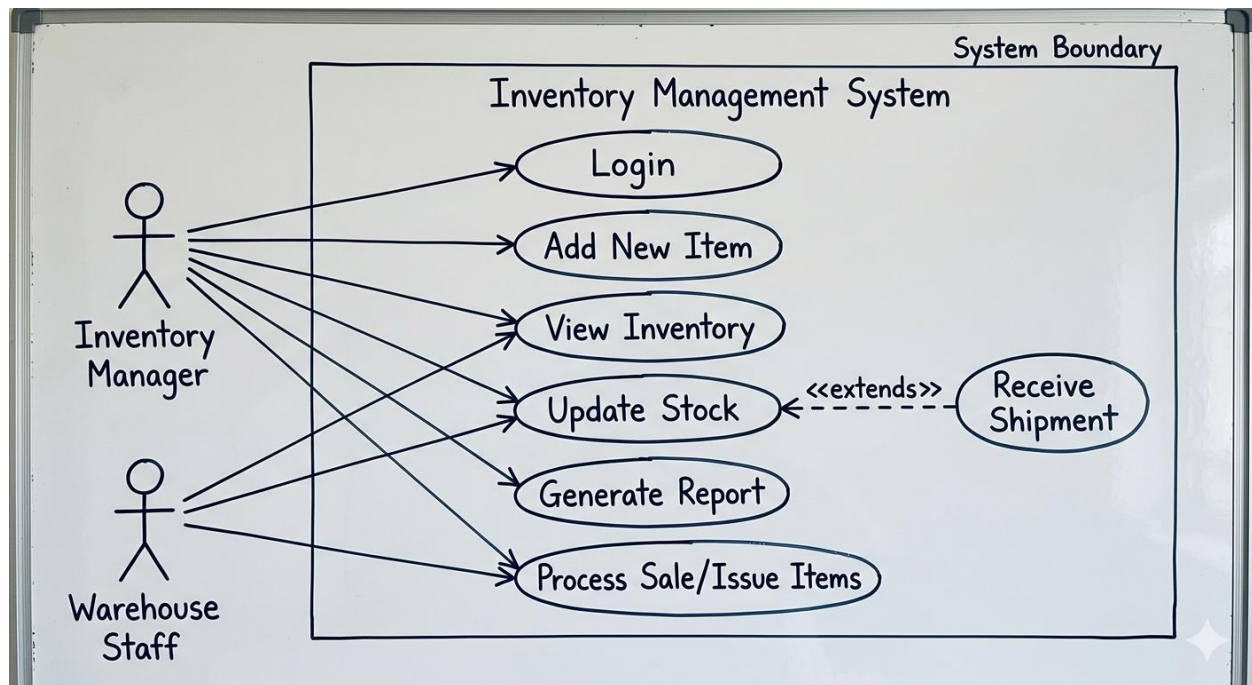
1. Use Case Diagram (Actors: 2; Use Cases: 5+)

Actors:

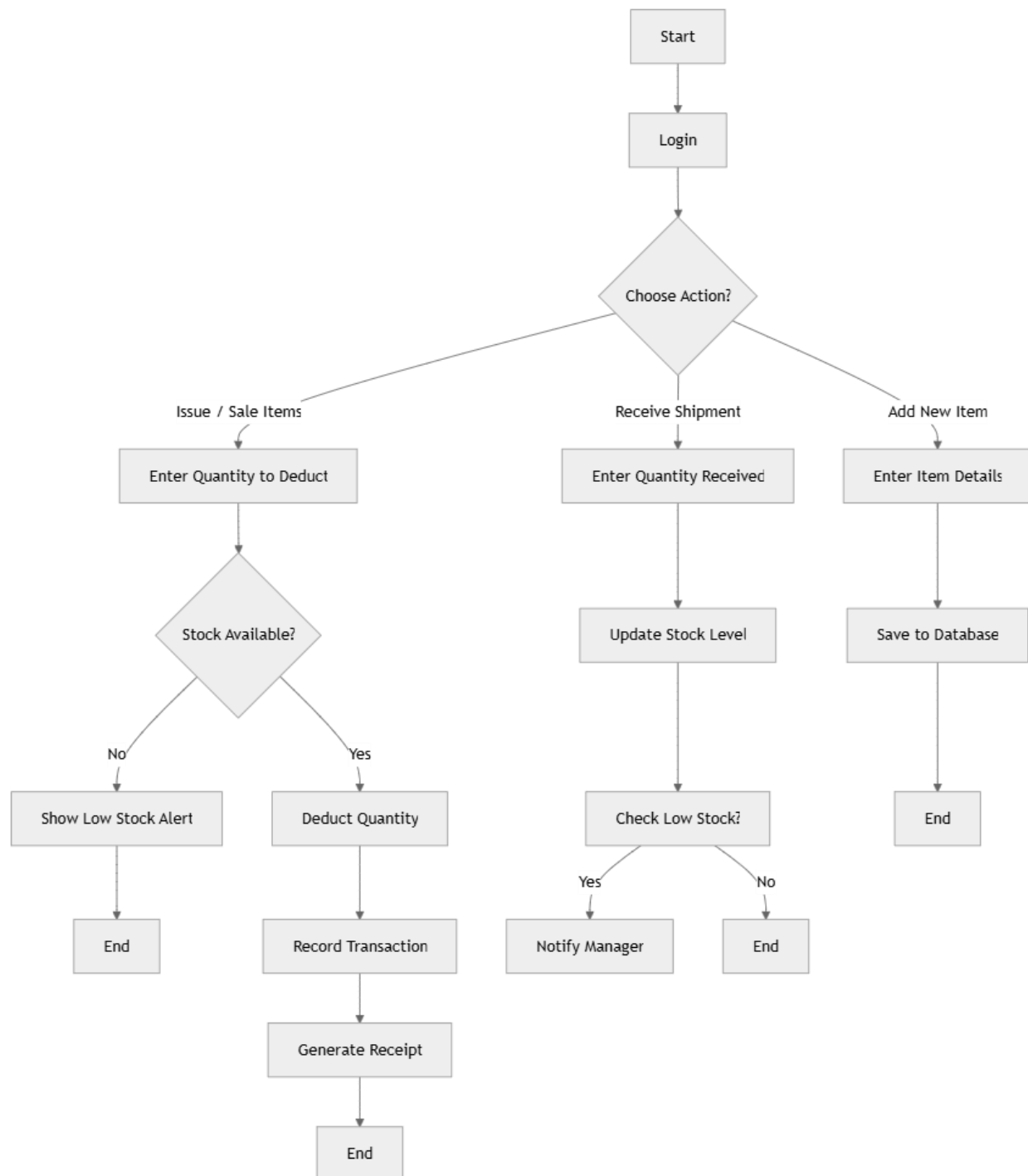
- **Inventory Manager** (primary actor)
- **Warehouse Staff**

Use Cases (ovals connected to actors with lines; include <<extends>> or <<includes>> where needed):

- Add New Item
- Update Stock
- View Inventory
- Generate Report
- Process Sale/Issue Items
- Login (common to both)
- Receive Shipment (extends Update Stock)



2. Activity Diagram



Part 3: SDLC Application

- **Planning:** Identify the need for an efficient inventory system to minimize losses from stock discrepancies. Gather requirements from stakeholders (managers and staff), define scope, budget, and timeline. Feasibility study confirms technical and economic viability.
- **Design:** Create system architecture, database schema (e.g., tables for items, transactions), and UML diagrams (Use Case, Class, Activity). Design user interfaces for easy navigation and define security roles.
- **Implementation:** Code the application using a suitable language (e.g., Python/Django, Java, or PHP) and database (MySQL/SQLite). Integrate features like stock updates and reports. Deploy on a server or cloud.
- **Testing:** Perform unit testing (individual functions), integration testing (modules together), and user acceptance testing. Check for bugs in stock calculations, security vulnerabilities, and edge cases like negative stock.

Part 4: Reflection

1. **What did you learn from creating UML diagrams?** I learned that UML diagrams provide a clear visual way to plan and communicate system requirements and processes before coding. They help spot missing features or logic errors early, making development more organized and less prone to mistakes.
2. **Why is planning (SDLC) important before building a system?** Planning through SDLC ensures the system meets user needs, stays within budget and time, and reduces costly rework later. It provides a structured approach that improves quality, risk management, and overall project success.